Name: Trần Anh Tuấn
Student ID: 24521943
Class: CS4323.Q12.CTTT

OS
LAB 4

CHECKLIST

1. Homework

	1	2
Explanation	☒	☒
Your code	☒	☒
Results and discussions	☒	☒

<u>Mark:</u>

2. Lab 4

1. Write a program time.cpp that measures the execution time of a shell command. The program will be executed with the syntax ./time <command> where <command> is the shell command you want to measure the execution time for.

Example:

```
$ ./time ls
time.c
time
Execution time: 0.25422
```

Answer:

Code:

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <iomanip>

using namespace std;

int main(int argc, char *argv[]) {
    if (argc < 2) {
        cerr << "Usage: " << argv[0] << " <command>" << endl;
        return 1;
    }

    struct timeval start, end;

    gettimeofday(&start, NULL);

    pid_t pid = fork();

    if (pid < 0) {
        cerr << "Fork failed!" << endl;
        return 1;
    } else if (pid == 0) {
        execvp(argv[1], &argv[1]);
        cerr << "Command execution failed" << endl;
        return 1;
    } else {
        wait(NULL);
```

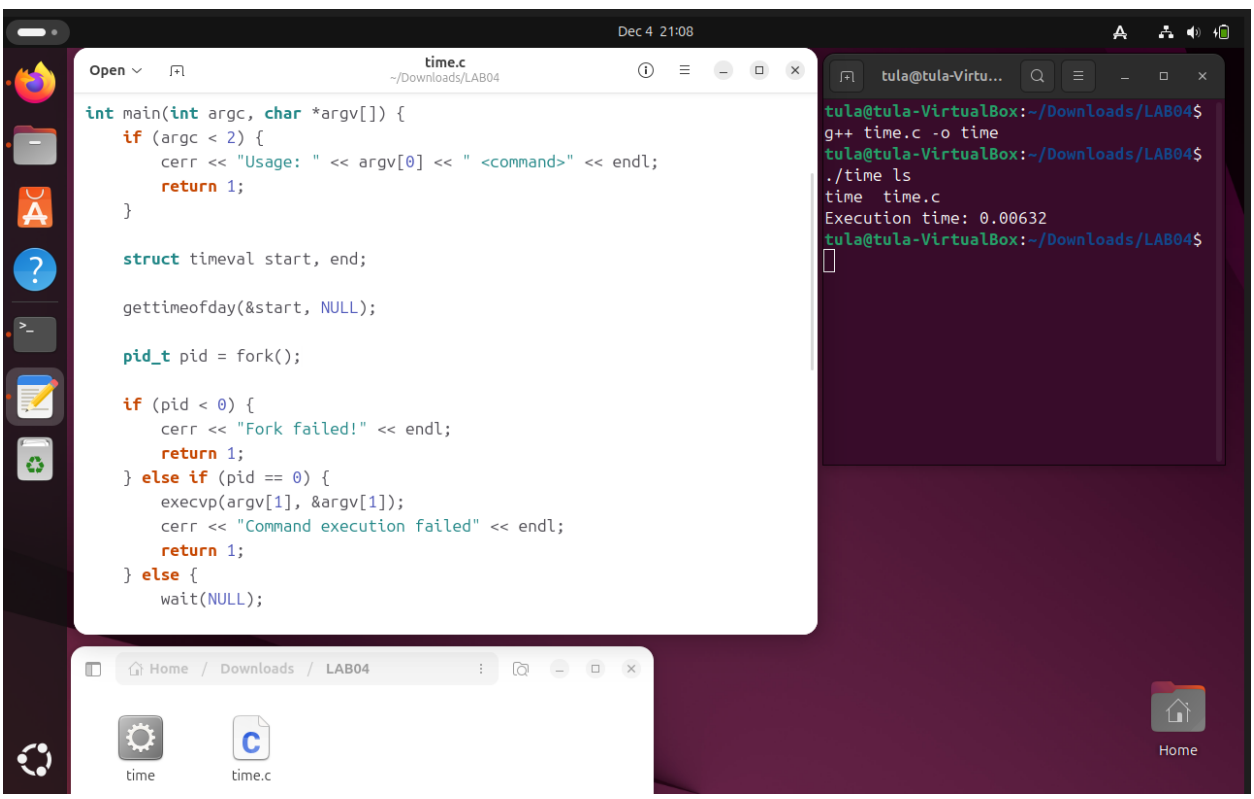
```
        gettimeofday(&end, NULL);

        double time_taken;
        time_taken = (end.tv_sec - start.tv_sec) * 1e6;
        time_taken = (time_taken + (end.tv_usec - start.tv_usec)) *
1e-6;

        cout << "Execution time: " << fixed << setprecision(5) <<
time_taken << endl;
    }

    return 0;
}
```

Result:



Explanation:

- **Start Timer:** The program records the current timestamp using `gettimeofday()`.
- **Create Process (`fork`)**: The program splits into two processes:

- **Child Process:** Executes the command provided by the user (e.g., `ls`) using `execvp()`. This replaces the child process code with the new command.
- **Parent Process:** Pauses and waits for the child to finish using `wait(NULL)`.

- **End Timer:** Once the child process finishes, the parent wakes up and records the ending timestamp.

- **Calculate:** It calculates the difference between the **end time** and **start time** to display the total execution duration in seconds.

2. Design a file-copying program named filecopy.cpp using ordinary pipes. This program will be passed two parameters: the name of the file to be copied and the name of the destination file. The program will then create an ordinary pipe and write the contents of the file to be copied to the pipe. The child process will read this file from the pipe and write it to the destination file. For example, if we invoke the program as follows:

./filecopy input.txt copy.txt

The file input.txt will be written to the pipe. The child process will read the contents of this file and write it to the destination file copy.txt.

Answer:

Code:

```
#include <iostream>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <cstring>

#define BUFFER_SIZE 4096

using namespace std;

int main(int argc, char *argv[]) {
    if (argc != 3) {
        cerr << "Usage: " << argv[0] << " <source_file>
<destination_file>" << endl;
        return 1;
    }
```

```
    const char* sourceFile = argv[1];
    const char* destFile = argv[2];

    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("Pipe failed");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid > 0) {
        close(pipefd[0]);

        int inputFd = open(sourceFile, O_RDONLY);
        if (inputFd == -1) {
            perror("Error opening source file");
            close(pipefd[1]);
            return 1;
        }

        char buffer[BUFFER_SIZE];
        ssize_t bytesRead;
        while ((bytesRead = read(inputFd, buffer, BUFFER_SIZE)) > 0)
{
            if (write(pipefd[1], buffer, bytesRead) != bytesRead) {
                perror("Error writing to pipe");
                break;
            }
        }

        close(inputFd);
        close(pipefd[1]);

        wait(NULL);
    }
    else {
        close(pipefd[1]);

        int outputFd = open(destFile, O_WRONLY | O_CREAT | O_TRUNC,
0666);
        if (outputFd == -1) {
            perror("Error opening destination file");
            close(pipefd[0]);
```

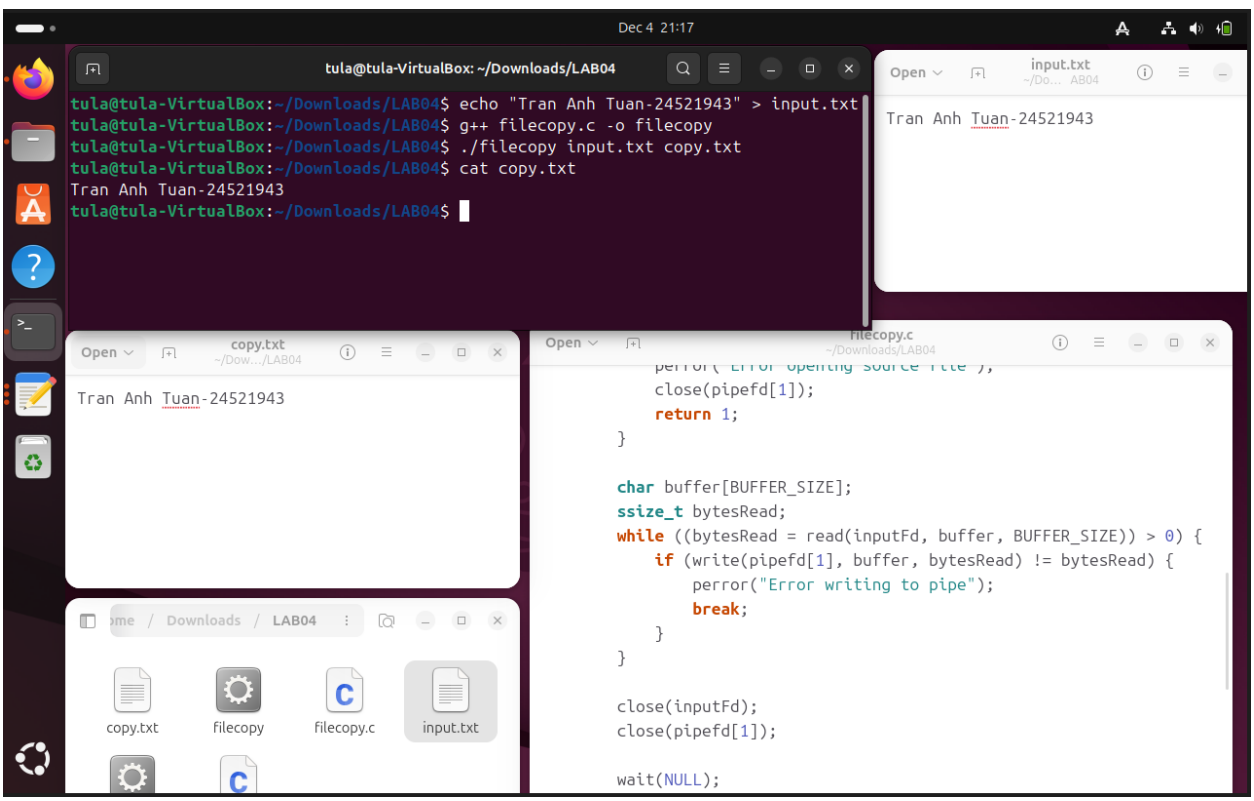
```
            return 1;
        }

        char buffer[BUFFER_SIZE];
        ssize_t bytesRead;
        while ((bytesRead = read(pipefd[0], buffer, BUFFER_SIZE)) >
0) {
            if (write(outputFd, buffer, bytesRead) != bytesRead) {
                perror("Error writing to destination file");
                break;
            }
        }

        close(outputFd);
        close(pipefd[0]);
    }

    return 0;
}
```

Result



Explanation:

- **Pipe Creation:** The program creates a pipe (`pipefd`) with a read end (`pipefd[0]`) and a write end (`pipefd[1]`).

- **Fork:** It calls `fork()` to create a child process.

- **Parent Process (The Sender):**

  - Reads data from the **source file**.
  - Writes that data into the **write end of the pipe** (`pipefd[1]`).
  - Closes the pipe to signal the end of data and waits for the child to finish (`wait`).

- **Child Process (The Receiver):**

  - Reads data from the **read end of the pipe** (`pipefd[0]`).
  - Writes that data into the **destination file**.

- **Result:** Data flows effectively as: **Source File → Parent → Pipe → Child → Destination File**.